2026 CSP-S 复赛模考 3 题解

T1 图灵机

【题目大意】

给定一个操作串 s，从空串开始依次执行操作：小写字母表示在末尾追加该字符，`-` 表示删除末尾字符（空串时不操作），`*` 表示把整个串复制并接到末尾，`!` 表示反转整个串。求所有操作完成后字符串的第 k 个字符；若长度不足 k，输出 `.`。

【参考难度】

提高。

【题目解析】

直接模拟是最容易想到的方法，但遇到 `*` 时长度会翻倍。题目虽然保证过程中字符串长度不超过 10^{18}，但字符串本身仍可能远大于可用空间，因此不能真的构造最终字符串。可以改为只追踪目标字符的位置。

先从左到右记录每一步操作结束后的字符串长度。若最终 k 大于最终长度，答案为 `.`；否则从最后一步开始逆向处理，维护目标字符在当前字符串中的位置 k。

设当前逆向处理的操作执行后，字符串长度为 L。

- 操作为小写字母 c：若 $k = L$，目标字符就是这一步追加的 c；否则它来自更早的字符串，继续向前追踪。
- 操作为 `-`：当前字符串是删除前字符串的前缀，位置 k 不变。
- 操作为 `*`：新串由原串及其副本组成。若 $k > L/2$，令 $k \leftarrow k - L/2$；否则位置不变。
- 操作为 `!`：反转前后的位置一一镜像，令 $k \leftarrow L - k + 1$。

逆向过程中一旦遇到追加字母且 $k = L$，即可确定答案。每个操作只处理一次。

测试点 $1 \sim 3$（15 分）：

$|s| \le 10$。可以直接构造字符串并执行全部操作，再判断第 k 个字符。

测试点 $1 \sim 3$、$9 \sim 11$（累计 30 分）：

特殊性质 A 为 s 中不含 `*`。此时可以用双端队列维护字符串，并用一个方向标记表示逻辑上的正向或反向；追加、删除分别在对应端操作，遇到 `!` 时只需切换方向。字符串长度不会因复制而增长，时间复杂度为 $O(|s|)$。

测试点 $1 \sim 3$、$9 \sim 14$（累计 45 分）：

特殊性质 B 为 s 中不含 `!`。在已有的小规模直接模拟和特殊性质 A 解法之外，还可以记录每一步的字符串长度，并从后向前追踪位置。追加、删除和复制的逆向处理均为常数时间，总时间复杂度为 $O(|s|)$。

正解（100 分）：

完整数据允许四种操作同时出现。正向扫描记录每一步后的长度，逆向扫描维护目标位置。复制操作中，若目标位置在后半段，就减去原串长度 $L/2$；反转操作使用 $k \leftarrow L - k + 1$；追加操作只有在当前位置恰为末尾时才是目标字符。

长度及位置使用 64 位整数即可，因为题目保证过程中长度不超过 10^{18}。正向记录长度和逆向追踪各扫描一次字符串，因此

$$\text{时间复杂度 } O(|s|), \qquad \text{空间复杂度 } O(|s|)。$$

T2 翻转纸牌

【题目大意】

有 n 张从左到右排列的纸牌，王牌初始位于第 p 张。每次选择一个长度恰为 k 的连续区间并将区间顺序翻转；有 m 个锁定位置，王牌不能停留在这些位置。对每个位置求王牌到达它所需的最少操作次数；不可达时输出 -1。

【参考难度】

提高。

【题目解析】

纸牌中只有王牌的位置会影响答案，因此可以把每个位置看作图上的一个状态。若一次合法翻转能把王牌从位置 i 移到位置 j，就在 i, j 之间连一条边。所有操作代价相同，答案就是从位置 p 出发的无权最短路，可以用 BFS 求出。

考虑一次翻转：若长度为 k 的区间左端点为 l，且王牌原来在位置 i，那么翻转后的位置为

$$j = 2l + k - 1 - i。$$

这里全部使用 1-based 下标。包含位置 i 的合法区间左端点满足

$$\max(1, i - k + 1) \le l \le \min(i, n - k + 1)。$$

代入位置转移式可知，j 以步长 2 取遍以下区间中的某一固定奇偶位置：

$$\max(i - k + 1, k - i + 1) \le j \le \min(i + k - 1, 2n - k - i + 1)。$$

并且 $j \equiv k - 1 - i \pmod 2$。因此，每个 BFS 状态虽然可能有很多种翻转方式，但所有候选目标位置都可以压缩成“一个区间中的固定奇偶位置”。锁定位置不能进入队列，也不能作为后续状态。

测试点 $1 \sim 2$（8 分）：

$n, k \le 5$。直接枚举所有可能的区间翻转并建立状态转移，再用 BFS 求最短距离。

测试点 $1 \sim 5$（累计 20 分）：

$n, k \le 2000$ 且 $m = 0$。朴素 BFS 对每个出队位置枚举所有合法区间，时间复杂度为 $O(nk)$。也可以利用后面的区间与奇偶结构加速。

测试点 $1 \sim 10$（累计 40 分）：

$n, k \le 2000$，锁定位置可能存在。对每个出队位置枚举合法区间，跳过锁定位置和已经访问的位置，时间复杂度为 $O(nk)$。

测试点 $11 \sim 16$（累计 64 分）：

这些测试点分别满足 $k = 1$、$k = 2$、$k = 3$、$k = n$ 或 $k = n - 1$。当 k 为常数时，每个位置对应的合法区间数至多为 k；当 $k = n$ 或 $k = n - 1$ 时，候选区间数也只有 1 或 2。因此朴素 BFS 可以处理这些特殊档位。

特殊性质（测试点 $17 \sim 20$，累计 80 分）：

当 $m = 0$ 时，可以按奇偶分别维护尚未访问的位置，并用“下一个未访问位置”并查集跳过已经入队的位置。处理一个状态时，只需取出其可达区间内、奇偶性符合要求的位置。每个位置最多被删除一次，BFS 总体可以做到近似线性时间。

正解（100 分）：

按 BFS 分层，用两个有序集合分别维护奇数位置和偶数位置中尚未访问且未锁定的位置。处理位置 i 时，先根据公式求出可达区间端点和目标奇偶性，再从对应集合中取出落在区间内的所有位置，逐个赋值为 $\mathrm{dist}[i] + 1$ 并入队，同时从集合中删除。

每个位置最多进入队列一次、从集合中删除一次。每次有序集合查找或删除花费 $O(\log n)$，故

$$\text{时间复杂度 } O(n \log n), \qquad \text{空间复杂度 } O(n)。$$

T3 未来程序

【题目大意】

给定长度为 n 的数组和 q 次操作。每次操作给出 (l, r, k, v)，将下标 $l, l + k, l + 2k, \cdots \le r$ 对应的元素乘以 v 并对 $10^9 + 7$ 取模。所有操作完成后，求数组元素的按位异或和。

【参考难度】

提高+/省选-。

【题目解析】

直接模拟每次操作，需要遍历 $l, l + k, l + 2k, \ldots$。当步长 k 很小时，一次操作可能更新很多元素；若操作数 q 也很大，重复更新会超时。

关键在于所有操作都是模意义下的乘法。乘法满足交换律，因此可以独立汇总每个位置最终受到的乘数，再乘到初始值上。按照 k 的大小分治：

- 大步长操作：每次最多影响约 n/k 个位置，可以直接枚举。

- 小步长操作：固定 k 后，按下标模 k 的余数分组。每个余数类中的下标构成步长为 k 的序列，操作在这个序列上就变成连续区间乘法，可以用乘法差分批量处理。

对乘法差分数组，在区间左端乘上 v，在右端之后乘上 v^{-1}。对差分数组做前缀乘积后，区间内位置会乘上 v，区间外的影响会抵消。模数 10^9+7 是质数，且 $1 \le v \le 10^9$，所以逆元存在，可用费马小定理计算 $v^{MOD-2} \bmod MOD$。

测试点 $1 \sim 2$（10 分）：

$n, q \le 1000$，并满足性质 C（每次操作 $v \le 5$）。在小规模下直接模拟全部操作即可。

测试点 $1 \sim 3$（累计 15 分）：

$n, q \le 1000$。直接模拟每次操作并最终计算异或和，复杂度为 $O\Big(n + \sum_{j=1}^{q} \Big(\Big\lfloor \frac{r_j - l_j}{k_j} \Big\rfloor + 1\Big)\Big)$。

特殊性质 A（测试点 $4 \sim 7$，累计 35 分）：

每次操作都有 $k > \lfloor \sqrt{n} \rfloor$。一次操作至多修改 $O(\sqrt{n})$ 个元素，因此直接逐项模拟的总复杂度为 $O(q\sqrt{n})$，最后用 $O(n)$ 计算异或和。

特殊性质 B（测试点 $8 \sim 11$，累计 55 分）：

每次操作 $k = 1$，操作退化为普通区间乘法。对每次区间乘法，在左端乘 v、右端后一位乘 v^{-1}，做一次前缀乘积即可还原每个位置的累计乘数。总体为 $O(n+q)$ 次差分更新与扫描（计算逆元另需快速幂）。

特殊性质 C（测试点 $12 \sim 16$，累计 80 分）：

每次操作 $1 \le v \le 5$。这一条件使乘数的取值范围较小，但并不限制每次操作会影响的位置数，因此不能仅凭 v 较小就把所有操作逐项展开。完整算法仍可利用小步长分组和乘法差分处理这些操作。

正解（100 分）：

令阈值 $B = \lfloor \sqrt{n} \rfloor + 1$。

当 $k > B$ 时，直接枚举本次操作修改的下标，一次操作最多影响 $O(n/B) = O(\sqrt{n})$ 个位置，所有大步长操作耗时 $O(q\sqrt{n})$。

当 $k \le B$ 时，按 $(k, l \bmod k)$ 分组。固定一组后，所有会被影响的位置构成一个步长为 k 的序列；把下标换成该序列中的序号，原操作就变为连续区间乘法。对区间左端乘 v、右端后一位乘 v^{-1}，最后对该组差分做前缀乘积并更新原数组。

对每个 k，所有余数类的长度总和为 $O(n)$；共有 $O(\sqrt{n})$ 种小步长，因此小步长部分为 $O(n\sqrt{n} + q)$。大步长操作为 $O(q\sqrt{n})$。若每次差分更新都用快速幂求逆元，还需 $O(q \log MOD)$；由于 $MOD = 10^9+7$ 固定，通常将其视为每次操作的常数开销。因此总时间复杂度记为

$$O((n+q)\sqrt{n})。$$

按组复用差分数组时，额外空间为 $O(n+q)$。

T4 计算

【题目大意】

给定一棵以 1 为根的有根树，每个节点有权值 w_i。叶子是输入节点，输入阶段会同时为所有叶子开辟大小为 w_i 的空间。非叶子是计算节点：只有等所有儿子都完成后才能计算；计算时先开辟大小为 w_i 的新空间，再释放所有儿子的空间。求一种计算顺序，使整个过程的峰值内存最小。

【参考难度】

省选/NOI-。

【题目解析】

叶子输入结束后，所有叶子的空间同时存在。之后，一个计算节点完成时，其儿子空间被释放，自己的空间保留。对同一个父节点而言，各个儿子子树的计算先后顺序可以调整；不同顺序会改变计算期间其他结果仍占用的空间，因此需要在满足树上依赖关系的前提下安排计算顺序。

考虑计算节点 x：在它完成计算的那一步，儿子结果都仍在内存中；申请 w_x 后，峰值可能增加，随后儿子空间被释放。可以用二元组 (al, fe) 概括一段计算过程：al 表示该段相对开始内存的峰值；fe 表示峰值与结束时内存变化量的差，即 $fe = al - \Delta$，其中 Δ 是该段结束内存减去开始内存。这样定义后，若先执行过程 x 再执行过程 y，合并后的状态为

$$\begin{aligned} al' &= \max(al_x,\ al_x - fe_x + al_y), \\ fe' &= al' - (al_x - fe_x + al_y - fe_y)。\end{aligned}$$

对一个初始计算节点 i，计算前其儿子结果占用 $\sum_{c\text{ 是 }i\text{ 的儿子}} w_c$，计算时新申请 w_i，完成后释放儿子空间。因此该节点对应的初始状态为

$$(al_i, fe_i) = \left(w_i,\ \sum_{c\text{ 是 }i\text{ 的儿子}} w_c \right)。$$

将这些过程按合适顺序合并，就能得到每个子树的状态。

测试点 $1 \sim 6$（30 分）：

$n \le 10$。可以枚举所有满足依赖关系的计算顺序，逐一计算峰值内存并取最小值。

测试点 $7 \sim 10$（20 分）：

$n \le 3 \times 10^3$，无特殊性质。后文的树上贪心与并查集方法适用于这一档及完整数据。

特殊性质 A（测试点 11）为所有节点均满足 $w_i = 1$；特殊性质 B（测试点 $12 \sim 15$）为每个非根节点 i 满足 $p_i = 1$ 或 $p_{p_i} = 1$。完整算法不依赖这两项性质。

正解（100 分）：

对每个计算节点维护一段子树计算过程的状态 (A, F)。若先执行组 x 再执行组 y，合并后的状态为

$$A' = \max(A_x, F_x + A_y), \qquad F' = F_x + F_y。$$

顺序的选择会影响峰值。可以按状态确定各组的优先级，反复选择当前优先级最高且尚未并入根的节点组，将它与父节点组的状态合并。并查集维护节点组当前对应的代表元；父节点组状态更新后重新加入优先队列。优先队列中可能存在已经过期的状态，弹出时需要检查它是否仍与当前状态一致，且对应节点仍为代表元。

所有非根节点组处理完成后，根组的峰值增量加上叶子输入时已经占用的空间，即为答案。记

$$W_{leaf} = \sum_{i\text{ 是叶子}} w_i\,,$$

则答案为

$$W_{leaf} + A_1。$$

每次合并只进行常数次并查集操作和一次优先队列插入、删除，至多合并 $n - 1$ 次，故

$$\text{时间复杂度 } O(n \log n), \qquad \text{空间复杂度 } O(n)。$$